

DCAS

MQTT Publisher

Version 2.9.1

User Guide

EC-Net 4 → MQTT Integration & Broker Testing Utility

Contents

1. Overview	3
2. Prerequisites	3
3. Interface Overview	3
4. Brokers Tab	5
5. Obix Tab	8
6. Topics Tab	10
7. End-to-End Workflow Summary	13
8. Using DCAS MQTT Publisher as a Standalone Broker Test Tool	13
9. Diagnostic Log & Troubleshooting	13
10. Importing and Exporting Topics	14

1. Overview

DCAS MQTT Publisher is a Windows desktop utility built to make EC-Net 4 to MQTT integrations fast to stand up and easy to maintain. Rather than writing and maintaining a custom bridge for every project, DCAS MQTT Publisher connects to the obix REST driver already built into EC-Net 4, reads live BACnet point data directly from the station, and republishes that data to one or more MQTT brokers on a schedule or whenever values change.

The application is organized into three tabs — Brokers, Obix, and Topics — that mirror the natural workflow of an integration: define where the data is coming from (Obix), define where it is going (Brokers), and define what gets published and how (Topics).

Two tools in one

In addition to its role as an obix-to-MQTT bridge, the Brokers tab can be used entirely on its own as a general-purpose MQTT connection tester — useful for validating a broker's address, port, TLS configuration, and credentials before an obix integration is even in the picture. See Section 8, “Using DCAS MQTT Publisher as a Standalone Broker Test Tool.”

2. Prerequisites

Before configuring an integration, confirm the following are in place:

- The obix driver is added and enabled on the target EC-Net 4 station, exposing the station's point tree over an HTTPS REST interface (e.g. under Drivers → ObixNetwork).
- An EC-Net user account exists that is dedicated to DCAS MQTT Publisher, and its authentication scheme is set to HTTPBasicScheme rather than the default digest or form-based scheme. DCAS MQTT Publisher authenticates to the obix endpoint using standard HTTP Basic Authentication, so a user configured with any other scheme will fail to connect even with correct credentials.
- Network reachability from the machine running DCAS MQTT Publisher to both endpoints: the EC-Net station's obix interface (HTTPS, typically the station's standard web port), and the target MQTT broker (commonly TCP 1883 for plain connections or 8883 for TLS).
- If the EC-Net station or the MQTT broker presents a self-signed or otherwise untrusted certificate, either install a trusted certificate or enable the corresponding “allow invalid/untrusted certificate” option described in Sections 4 and 5 below.
- Credentials for the MQTT broker (username/password and/or a client certificate), if the broker requires authentication.

3. Interface Overview

The application window is divided into three tabs, plus a Diagnostic Log panel that is visible at the bottom of every tab.

Tab	Purpose	Details
Brokers	Define, edit, and test connections to one or more MQTT brokers.	See Section 4
Obix	Connect to an EC-Net station's obix REST interface, browse/fetch live point data, and stage points for publishing.	See Section 5
Topics	The publish list — maps a Topic/Subtopic and payload to a broker, with QoS, retain, and scheduling controls.	See Section 6

Diagnostic Log

Every tab shares the same Diagnostic Log panel at the bottom of the window. It timestamps connection attempts, obix responses, and publish activity (e.g. “Obix response received: 336 characters” or “Published 6 changed topics”), and is the first place to look when something isn't behaving as expected. Use Copy to copy the full log to the clipboard, and Clear to reset the entry counter. A status line beneath the log (e.g. “Topic changes saved.”) reflects the result of the most recent action.

4. Brokers Tab

The Brokers tab is where MQTT broker connections are defined. Each broker you add becomes available for selection on the Topics tab, so a single instance of DCAS MQTT Publisher can publish the same or different points to multiple brokers simultaneously (for example, a local test broker and a cloud broker).

4.1 Broker Configuration

Field	What it does	Example
Name	A friendly label used to identify this broker throughout the rest of the application, including the Broker dropdown on the Topics tab. Should be unique and descriptive.	HiveMQ
Address	The hostname or IP address of the MQTT broker.	7ad5836d...s1.eu.hivemq.cloud
Port	The TCP port the broker is listening on. Common values are 1883 (unencrypted) and 8883 (TLS).	8883
Use TLS / SSL	Encrypts the connection to the broker. Should be enabled for any broker reachable over the public internet, and is required by most cloud MQTT providers (e.g. HiveMQ Cloud).	Checked

4.2 Authentication, Certificates and Advanced Options

This collapsible section holds broker-level authentication and MQTT protocol options.

Field	What it does	Example
Username / Password	Login credentials for the broker, if it requires username/password authentication.	admin
Allow untrusted / self-signed certificate	Skips certificate validation when Use TLS/SSL is enabled. Useful for lab brokers or self-signed test certificates; leave unchecked for production brokers with a valid certificate chain.	Unchecked
Client cert (.pfx) / Browse...	Path to a PKCS#12 (.pfx) client certificate bundle, for brokers that require mutual TLS (client certificate authentication) instead of, or in addition to, username/password.	(blank if not used)
Cert password	The passphrase protecting the selected .pfx file.	(blank if not used)
Client ID	The MQTT client identifier presented to the broker on connect. If left blank, DCAS MQTT Publisher generates one	dcas-ecnet-site1

Field	What it does	Example
	automatically. Set this explicitly when multiple clients/integrations connect to the same broker, since a duplicate client ID will cause the broker to disconnect one of the sessions.	
Clean session	When checked, the broker discards any prior session state (queued messages, subscriptions) each time this client connects. Leave checked for a pure publisher unless you specifically need session persistence across reconnects.	Checked
Keep alive (s)	How often, in seconds, the client pings the broker to keep an idle connection open. Lower values detect a dropped connection sooner at the cost of slightly more network traffic.	60
Last Will topic	The MQTT topic the broker will publish to automatically if this client disconnects ungracefully (e.g. network loss, crash). Commonly used for online/offline status.	status/dcas/site1
Last Will payload	The message body published to the Last Will topic on an unexpected disconnect.	offline
Will QoS	The Quality of Service level (0, 1, or 2) used when the broker sends the Last Will message.	0
Will retain	When checked, the broker retains the Last Will message so it is immediately delivered to any client that subscribes after the fact.	Unchecked

4.3 Broker List Actions

Action	What it does	Example
Add Broker	Saves the values currently entered above as a new row in the broker list.	—
Update Selected	Applies the currently entered field values to whichever broker row is selected in the list below, without creating a new entry.	—
Remove Selected	Deletes the selected broker from the list. Any topics on the Topics tab pointing at it will need to be reassigned.	—
Test Connection	Attempts to open a connection to the broker using the values currently entered in the fields above — without needing to Add or Save first. Success or failure, along with any error detail (auth rejected, TLS handshake failure, timeout, etc.), is written to the Diagnostic Log.	—

The broker list beneath these buttons summarizes every configured broker (Name, Address, Port, TLS, Username, Client ID, Clean, Keep Alive). Selecting a row loads its values back into the fields above for editing.

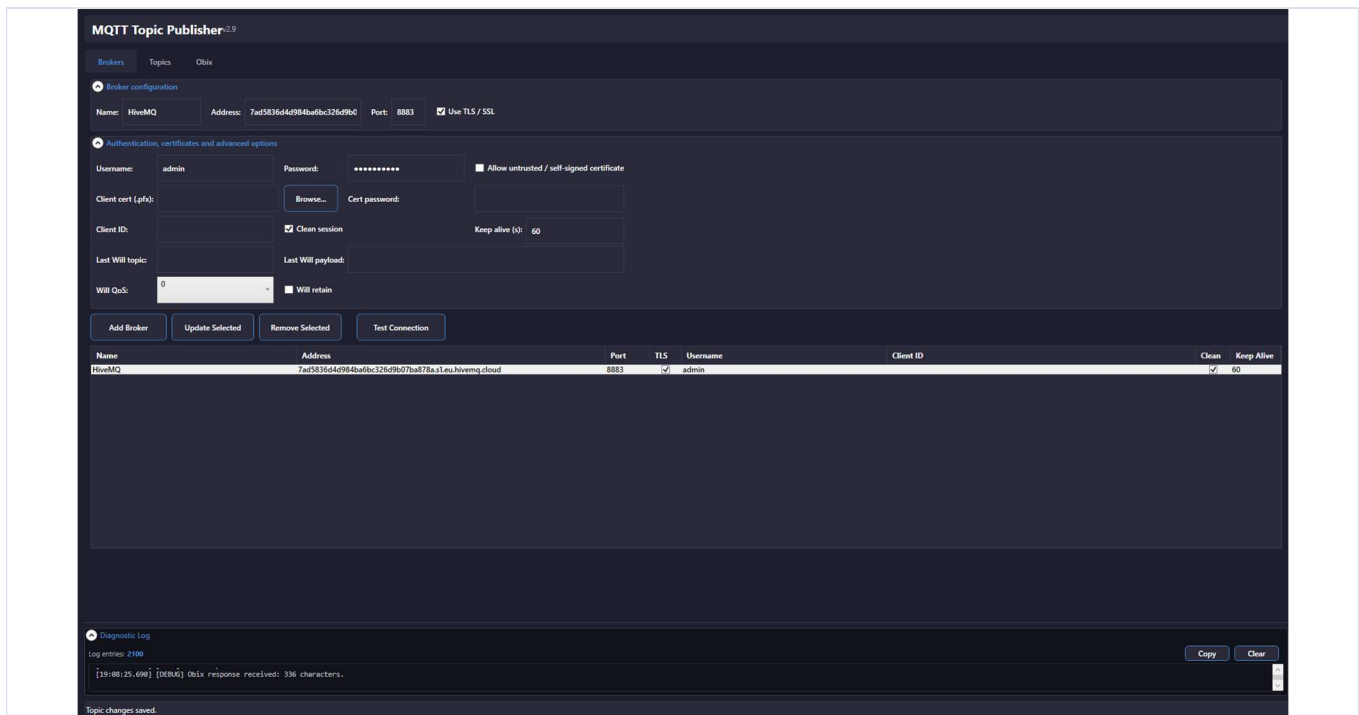


Figure 1 — Brokers tab, configured for a TLS connection to a HiveMQ Cloud broker.

4.4 Example: Configuring a Cloud Broker

To connect to an MQTT broker such as HiveMQ Cloud, as shown in Figure 1:

1. Enter a Name to identify the broker, e.g. HiveMQ.
2. Enter the broker's Address (the hostname provided by the broker vendor) and its Port — 8883 for a TLS listener.
3. Check Use TLS / SSL, since the connection is over the public internet.
4. Expand Authentication, certificates and advanced options and enter the Username and Password issued for the broker.
5. Leave Client cert (.pfx) blank unless the broker specifically requires mutual TLS.
6. Leave Clean session checked and Keep alive (s) at its default of 60 unless you have a specific reason to change them.
7. Click Test Connection and confirm success in the Diagnostic Log before clicking Add Broker to save it.

5. Obix Tab

The Obix tab is where DCAS MQTT Publisher connects to the EC-Net 4 station's obix REST interface, browses the exposed point tree, and pulls in live values that can then be handed off to the Topics tab for publishing.

5.1 Obix Connection Settings

Field	What it does	Example
Obix URL	The full HTTPS REST path into the EC-Net station's point tree, exposed by the obix driver. Pointing this at a specific network or device level (rather than the station root) scopes what Fetch Data retrieves — e.g. targeting a single BACnet device under a BACnet network returns just that device's points.	<code>https://192.168.2.20/obix/config/Drivers/BcpBacnetNetwork/ECB_600</code>
Username / Password	Credentials for the EC-Net user account dedicated to this integration. This account must be configured with the HTTPBasicScheme authentication scheme (see Section 2) or the connection will be rejected.	obix
Base Topic	A prefix that is combined with each discovered point's path when it is sent to the Topics tab, letting you namespace everything coming from this obix connection under a common root.	obix
Allow invalid certificate	Skips certificate validation for the obix HTTPS connection. Needed when the EC-Net station is using its default self-signed certificate rather than one issued by a trusted CA.	Checked

5.2 Obix Actions

Action / Setting	What it does	Example
Fetch Data	Queries the Obix URL and populates the results table below with every point discovered at that level of the tree, along with its current live value.	—

Action / Setting	What it does	Example
Send Selected to Topics	Takes the rows currently selected in the results table and creates (or updates) matching entries on the Topics tab, wiring each one to its Live Obix Source so its payload can be kept in sync automatically.	—
Save Settings	Persists the Obix URL, credentials, and Base Topic so they are pre-filled the next time DCAS MQTT Publisher is opened.	—
Poll interval (sec)	Sets how frequently, in seconds, Start Auto-Update re-queries the obix URL for fresh values.	5
Start Auto-Update / Stop Auto-Update	Toggles continuous polling of the Obix URL at the configured Poll interval, refreshing the Payload column live as station values change.	—

The results table lists every point returned by the most recent Fetch Data: its Topic and Subtopic (as they will appear if sent to the Topics tab), the full obix API Path used to read it, its current Payload, and its data Type (e.g. bool, ref).

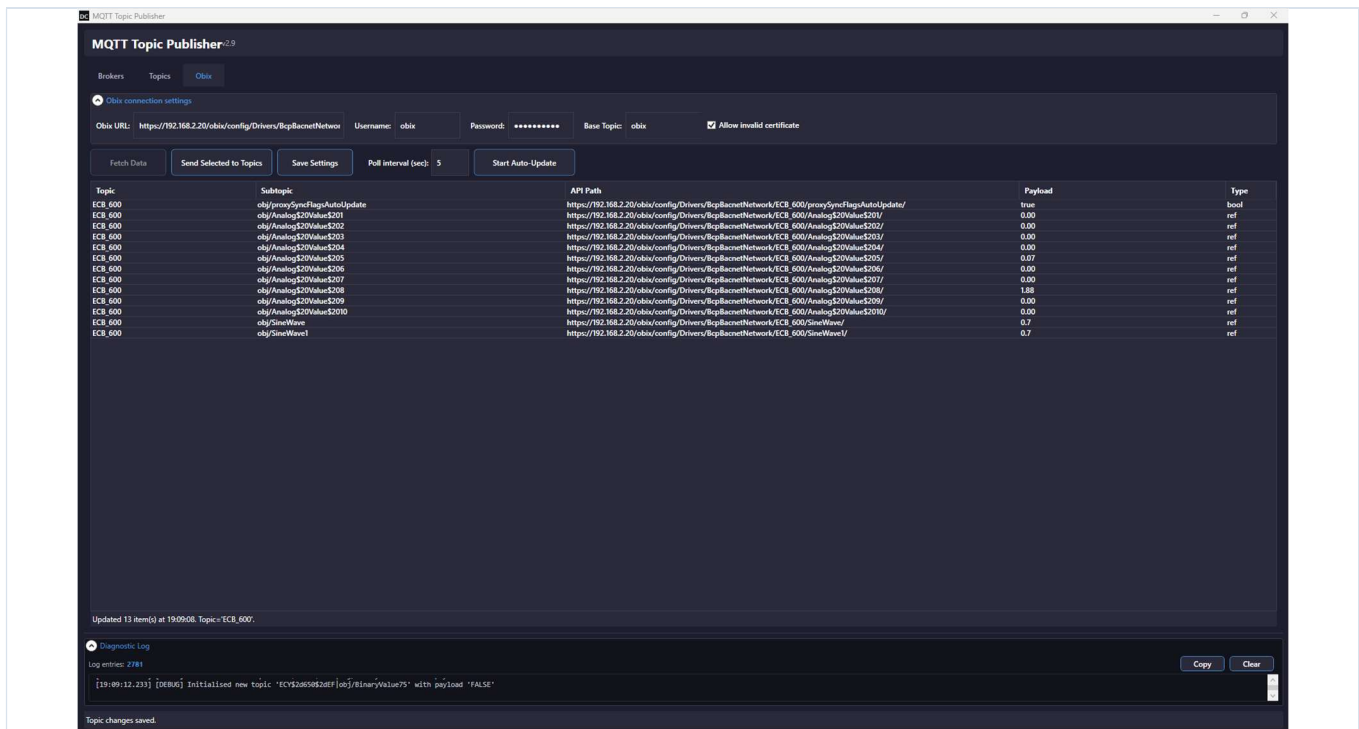


Figure 2 — Obix tab, connected to an EC-Net station and auto-updating a BACnet device's points.

5.3 Example: Pulling Points from a BACnet Device

To browse and stage the points of a single BACnet device (ECB_600) behind the BcpBacnetNetwork driver, as shown in Figure 2:

1. Set Obix URL to the device-level path, e.g.
`https://192.168.2.20/obix/config/Drivers/BcpBacnetNetwork/ECB_600/`.
2. Enter the obix Username and Password for the dedicated EC-Net account.
3. Set Base Topic to a value that identifies this station or device, e.g. obix.

4. If the station is using a self-signed certificate, check Allow invalid certificate.
5. Click Fetch Data. The results table populates with each point under that device — analog values, proxy sync flags, and so on — along with its live Payload.
6. Select the rows you want to publish and click Send Selected to Topics.
7. Optionally set a Poll interval (sec) and click Start Auto-Update to keep the Payload column refreshed while you continue working.

6. Topics Tab

The Topics tab is the publish list: every row pairs a Topic/Subtopic and a payload with a destination broker, and controls how and when that payload actually gets sent.

6.1 Topic / Subtopic Fields

Field	What it does	Example
Topic	The top-level MQTT topic segment. When a point is sent here from the Obix tab, this is typically populated from the device or station name.	ECY\$2d650\$2dEF
Subtopic	The remainder of the MQTT topic path, appended after Topic. When populated from the Obix tab it mirrors the point's path within the device.	obj/Analog\$20Value\$2019
Broker	A dropdown listing every broker configured on the Brokers tab. Selects which broker this row publishes to.	HiveMQ
QoS	The MQTT Quality of Service level (0, 1, or 2) used when publishing this specific topic.	0
Retain	When checked, the broker retains the last published message on this topic so it is delivered immediately to any new subscriber.	Unchecked
Payload	The value that will be published. This can be typed in manually for a static/test topic, or is kept up to date automatically for topics linked to a Live Obix Source.	0.00

6.2 Publishing and List Actions

Action / Setting	What it does	Example
Add Topic	Creates a new row using the values currently entered in the fields above.	—
Update Selected	Applies the currently entered field values to whichever row is selected in the table.	—
Remove Selected / Remove All	Deletes the selected row, or clears the entire topic list.	—
Publish Selected / Publish All	Immediately publishes the payload of the selected row(s), or every row, to its assigned broker — useful for a one-off manual push or for confirming a topic is wired correctly.	—

Action / Setting	What it does	Example
Interval (sec)	The publish cadence, in seconds, used by the interval publishing loop described below.	10
Auto-start on launch	When checked, interval publishing begins automatically as soon as DCAS MQTT Publisher opens, without needing to click Start.	Unchecked
Auto-publish changed values (grouped by URL)	When checked, the application watches each topic's Live Obix Source and republishes automatically only when the underlying value actually changes, batching updates that share the same obix URL into a single publish pass rather than blindly re-sending everything on a timer.	Checked
Start / Stop Interval Publishing	Toggles the scheduled publish loop that republishes topics on the configured Interval (sec), independent of the change-detection behavior above.	—
Export CSV... / Import CSV...	Saves the entire topic list to a CSV file, or loads one back in — useful for backing up a mapping, reusing a topic set across similar sites, or bulk-editing outside the application.	—

The table at the bottom of the tab lists every configured topic: Topic, Subtopic, Payload, Broker, QoS, Retain, and — for topics created from the Obix tab — the Live Obix Source, which shows the full obix API path feeding that row's payload.

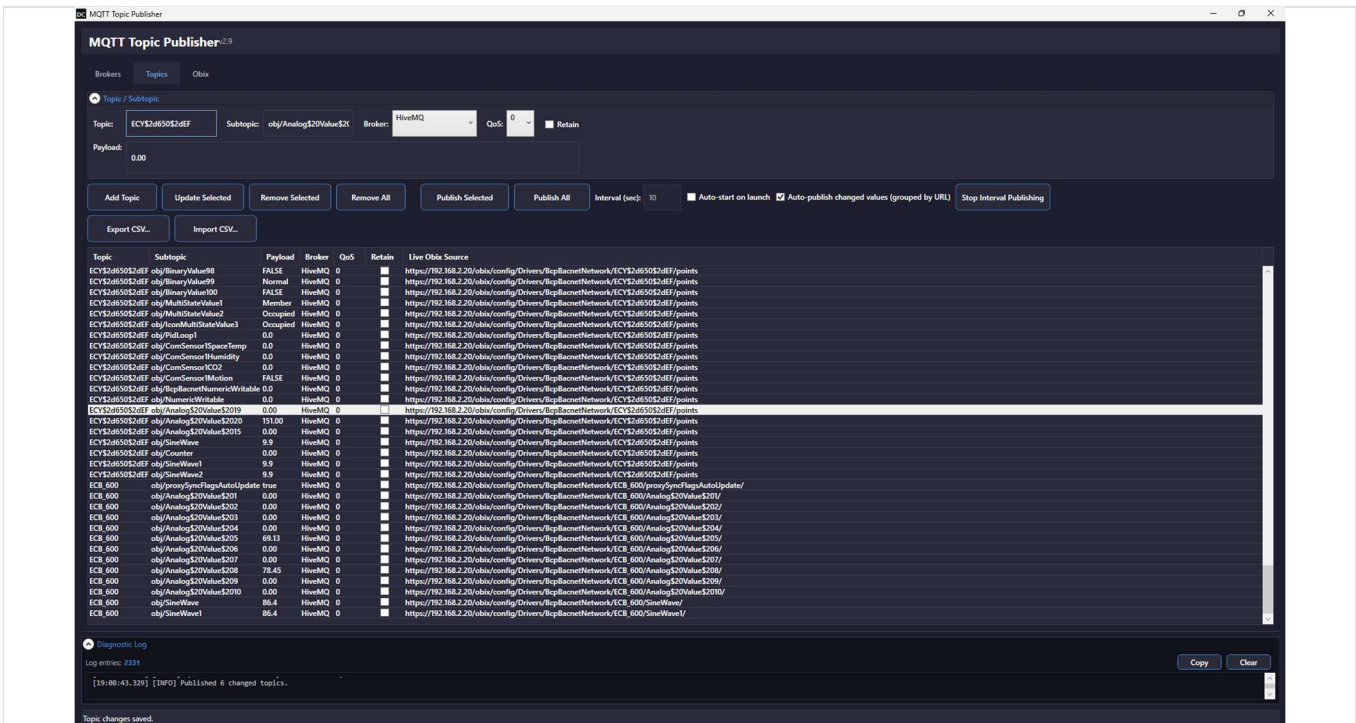


Figure 3 — Topics tab, with points from two BACnet devices linked to their Live Obix Source and publishing to HiveMQ.

6.3 Example: Publishing Fetched Points on a Schedule

Continuing from the Obix tab example in Section 5.3:

1. After clicking Send Selected to Topics, confirm each new row shows the expected Broker (e.g. HiveMQ) and a Live Obix Source pointing back at the correct obix path.
2. Adjust QoS and Retain per row if a given point needs different delivery guarantees than the rest — for example, enabling Retain on a status point so new subscribers immediately see its last value.
3. Check Auto-publish changed values (grouped by URL) so points only publish when their value actually changes, rather than flooding the broker with unchanged data.
4. Alternatively, or in addition, set an Interval (sec) and click Start Interval Publishing to republish on a fixed schedule.
5. Use Publish Selected on an individual row at any time to manually verify a specific topic without waiting for the next scheduled cycle.

7. End-to-End Workflow Summary

A typical first-time setup, from a blank configuration to a live, self-updating integration:

1. Confirm the obix driver is active on the target EC-Net 4 station and that a dedicated user with the HTTPBasicScheme authentication scheme exists (Section 2).
2. On the Brokers tab, add each MQTT broker the integration needs and click Test Connection to confirm reachability and credentials before relying on it (Section 4).
3. On the Obix tab, enter the station's Obix URL and credentials, then Fetch Data to browse the available points (Section 5).
4. Select the points to publish and click Send Selected to Topics.
5. On the Topics tab, assign or confirm the Broker, QoS, and Retain for each topic (Section 6).
6. Use Publish Selected / Publish All to confirm everything is reaching the broker as expected.
7. Enable Auto-publish changed values and/or Start Interval Publishing to hand the integration off to run unattended.
8. Periodically check the Diagnostic Log for connection or publish errors, and use Export CSV to keep a backup of the topic mapping.

8. Using DCAS MQTT Publisher as a Standalone Broker Test Tool

The Brokers and Topics tabs work independently of the Obix tab, which makes DCAS MQTT Publisher a convenient general-purpose MQTT test client — for example, to validate a customer's broker before any EC-Net integration work begins, or to confirm a subscriber-side application is receiving messages correctly.

- Add the broker under test on the Brokers tab and click Test Connection to verify its address, port, TLS/certificate configuration, and credentials in isolation.
- On the Topics tab, use Add Topic to create one or more test topics with a manually typed Payload, select the broker under test, and click Publish Selected to send a one-off message.
- Set Retain on a test topic to confirm retained-message behavior, or configure a Last Will topic/payload on the Brokers tab to test online/offline detection logic on the subscriber side.

9. Diagnostic Log & Troubleshooting

The Diagnostic Log at the bottom of the window timestamps every significant action and is the primary place to look when a connection, fetch, or publish doesn't behave as expected.

Field	What it does	Example
Obix response received: N characters	The Obix URL was queried successfully and returned a response of the given size. Confirms the obix driver, URL, and credentials are working.	Informational
Initialised new topic '<name>' with payload '<value>'	A new row was created on the Topics tab, typically as a result of Send Selected to Topics.	Informational
Published N changed topics	The auto-publish (change-detection) or interval publishing loop evaluated the topic list and sent N updated payloads to their brokers.	Informational
Topic changes saved.	Status line confirming the current topic list was written to disk.	Informational

If Fetch Data or Test Connection fails, check the log entry for the specific cause — typically an authentication rejection (verify the EC-Net user's authentication scheme is HTTPBasicScheme), a certificate error (toggle the relevant “allow invalid/untrusted certificate” option), or a timeout (verify network reachability and that the obix driver is enabled and running).

Use the Copy button to copy the full log to the clipboard when escalating an issue, and Clear to reset the log and entry counter.

10. Importing and Exporting Topics

The Export CSV... and Import CSV... buttons on the Topics tab read and write the topic list as a CSV file, with one row per topic matching the columns shown in the Topics table: Topic, Subtopic, Payload, Broker, QoS, Retain, and Live Obix Source.

- Export CSV... is useful for keeping an offline backup of a site's topic mapping, or for carrying a mapping over to a similar site as a starting point.
- Import CSV... loads a previously exported (or hand-built) CSV back into the Topics list. The Broker column values must match the Name of a broker already configured on the Brokers tab.

Document Information

Field	Value
Application	DCAS MQTT Publisher
Version covered	2.9.1
Audience	Distech Controls Advanced Support / field commissioning staff